

TEAM HANDBOOK & JUDGE BRIEFING

SmartMenu

Photograph a menu in any language. Get it translated, nutrition-scored and ranked against your diet. One photo, no typing.

1

MODEL CALL

8–11s

PHOTO TO MENU

2,654

LINES OF TS

18

MERGED PRS

How to use this document. Pages 2–3 are the story you tell a judge. Pages 4–7 are how the system actually works, so you can answer follow-ups. Pages 8–10 are the numbers, the failure plan, and the questions we expect to be asked.

Live: smartmenu-two-ruddy.vercel.app · Repo: github.com/mohamedhiba/smartmenu

Built by Mohamed Hiba, Abigail Vincent and Brandon Ljikovic.

1 • The problem

You are in Rome. The menu is in Italian. You are keto.

Google Translate gives you *words*. It tells you that *trofie al pesto* is “trofie with pesto”. It does not tell you that it is 76 grams of carbohydrate, or that pesto contains pine nuts even though the menu never says so.

Anyone eating to a rule — keto, coeliac, vegan, a nut allergy — is doing two translations at once. The language, and then the ingredients. The second one is the one that actually matters, and it is the one no tool does.

What SmartMenu does instead

1. You photograph the menu.
2. You tap your diet and any restrictions. Three taps.
3. Every dish comes back translated, with estimated macros, allergen flags, a score out of 100, and a plain-English reason for that score.

The sentence to land on: the ranking is not a translation, it is a decision. We do not tell you what the dish is called. We tell you whether to order it, and why.

Real output, from a real photo

A photographed Spanish menu (39 dishes, laminated, glare), read as `es`, ranked for keto:

SCORE	DISH	WHY
86	Cod in Tomato Sauce — Bacalao con tomate	Keto friendly
83	Tuna Salad — Ensalada con atún	Keto friendly
82	Assorted Cheese Board — Tabla de quesos	Keto friendly
23	Fried Peppers and Potatoes	38g carbs – not keto
19	Fried Breadcrumbs with Pork — Migas	50g carbs – not keto

Note what the model inferred that the menu never stated: that *migas* is bread-based and therefore not gluten-free, and that *tabla de quesos* is low enough in carbohydrate to suit keto. No database knows that. That inference is the product.

2 · The one decision worth defending

If a judge only remembers one technical thing, make it this. Everything else is execution.

THE OBVIOUS BUILD

```
photo → Google Vision OCR → GPT structuring → Google Translate  
      → FoodData Central lookup → scoring → UI
```

Five APIs, five sets of credentials, five failure points — and a nutrition lookup that has to fuzzy-match “Trofie al Pesto Genovese” against a database keyed on “pasta, cooked”. That last step alone is a research project.

WHAT WE BUILT

```
photo → downscale in browser → ONE multimodal call (strict JSON schema)  
      → validate → score → ranked menu
```

Why this is the right architecture, not a shortcut

- **A vision model reading a menu does not need OCR as a separate step.** It reads the image directly. Handing it pre-extracted text would actually *lose* information — layout is what tells you which line is a section header and which is a dish.
- **Translation is free once the model is already reading.** Asking for output in the target language costs nothing extra.
- **Ingredient inference is something only a language model can do.** No database knows carbonara contains egg and pork when the menu just says “Spaghetti alla Carbonara”.
- **Nutrition estimation is honest about what it is.** A restaurant does not publish macros. A lookup on a fuzzy foreign dish name would be an estimate too — just slower, wearing a lab coat.

The half that is deliberately not AI. Scoring is a pure function in `lib/scoring.ts`. The same dish and the same preferences always produce the same score, and the score can explain itself.

The model does perception. Deterministic code does judgement. That is why the app can tell you *why*, and why it cannot quietly change its mind between two people asking the same question.

What it proved it can read

On an Argentine menu that marks gluten-free dishes with a crossed-wheat *icon* rather than text, the model flagged exactly the five marked dishes and correctly did not flag the pasta, the pastry or the empanadas. It read a symbol, not a word.

3 · How a photo becomes a ranked menu



Why the image is downscaled in the browser

A phone photo is 3–5MB, and base64 inflates it by a third. **Vercel rejects request bodies over 4.5MB before our code runs.** Without this step the first real photo on stage returns 413 and the demo dies. It also cuts model latency and token cost by roughly five times.

Why the processing screen never navigates on a timer

An early version advanced its progress stages on a timer and pushed to the menu after ~4 seconds. The real call takes 8–20 seconds, so the user was shown the *sample* menu instead of their own photo, and an error could never appear because the page had already left. The stages are decorative; only the result drives navigation.

4 • The codebase

2,654 lines of TypeScript. Eight runtime dependencies. No database.

FILE	LINES	WHAT IT DOES
<code>lib/schema.ts</code>	131	The contract. zod definitions of <code>Prefs</code> , <code>MenuItem</code> , <code>AnalyzedMenu</code> , <code>Scored</code> . Everything else is downstream of this file.
<code>lib/llm.ts</code>	640	The model call. Prompt, generated response schema, key rotation, model and provider fallback, timeout budget, repair pass, nutrition sanity.
<code>lib/scoring.ts</code>	168	Deterministic ranking. Hard diet blocks, soft preferences, reason chips, tiebreaks.
<code>lib/store.ts</code>	160	zustand state. Calls the API, maps every error code to something a human can read.
<code>lib/image.ts</code>	53	Canvas downscale and base64 encode, in the browser.
<code>lib/fixtures.ts</code>	106	The 8-dish demo menu. Backs every screen and <code>?demo=1</code> .
<code>components/types.ts</code>	61	The UI contract. The props each screen takes.
<code>components/**</code>	14 files	Every visual. Primitives, five screens, <code>DishCard</code> , <code>ScoreBadge</code> , <code>CategoryTile</code> .
<code>app/**/page.tsx</code>	5 files	Thin containers. Own data, routing and state; no visuals.
<code>app/api/analyze</code>	113	The one endpoint that matters. Never returns a blank 500.
<code>app/api/ask-dish</code>	171	Follow-up questions about a dish.

The two contracts, and why they mattered

Three people built this in parallel. That worked because two files were agreed *before anyone wrote a screen*:

- `lib/schema.ts` fixes the shape of the data.
- `components/types.ts` fixes the props each screen takes.

Abigail built every component against those props. Brandon built the pages and state against the same props. Neither opened the other's files. When the two halves were finally connected, **the contract needed zero changes** — they fit on the first try.

The rule: pages own data, routing and state. Components own everything visible. A page should not contain a Tailwind class beyond layout.

5 · How the scoring works

This is the part judges probe hardest, because it is the part that makes a recommendation rather than a translation. It is ordinary code — no model involved — and that is the point.

Hard blocks come first

A dish that breaks the diet, or trips a declared allergy, is capped at “**limit**” no matter what else it has going for it. A veto, not a penalty. Someone who cannot eat gluten is not helped by a coeliac-unsafe dish scoring 60 out of 100.

But blocked dishes still rank against each other

On a menu where nothing fits the diet, an early version pinned every dish to exactly 35 — twelve identical red cards in arbitrary order, with the dish closest to acceptable buried in the middle. That is worse than useless.

Blocked dishes now spread across the band below “limit” by *how badly* they break the rule. Keto has a natural gradient in carbohydrate. Binary diets have no honest gradient, so calorie load is used purely as a tiebreak. An allergy is not a spectrum — those sink to the bottom.

KETO, SAME MENU	BEFORE	AFTER
Prawn & Brie Salad (16g carbs)	35 limit	33 — ranked first
Beef Filet (24g)	35 limit	30
Empanadas (38g)	35 limit — <i>ranked above the salad</i>	23
Ossobuco Sorrentinos (70g)	35 limit	7

Reasons are written for humans

`reasons []` is what the details screen shows under “Why this score”. They read as things a person would say — “*38g carbs – not keto*”, “*Contains gluten*”, “*High protein (46g)*” — not as field names. Negatives matter as much as positives: a coeliac needs to read “contains gluten”, not the absence of a green tick.

Nutrition sanity

The model estimates macros, so it occasionally produces nonsense — a 5000 kcal salad, a steak with no protein. Those are what a judge notices instantly, and scoring reads the numbers directly, so one absurd value drags a dish to the top or bottom for no reason. Implausible macros (out of range, or not adding up to the stated calories) are rejected and re-asked once.

6 · Why it does not fall over

Every request runs inside **one 40-second wall-clock budget**. Inside it, this ladder:

TIER	WHAT	FIRES WHEN
1	3 API keys, rotated from a moving start point	Always. Rotating the start spreads load instead of draining key 1.
2	gemini-3.6-flash , a different model	Keys spent, model retired, or contract broken. Rate limits are per model, so this is a different bucket.
3	OpenAI gpt-4o	Same, when a funded key exists.
—	?demo=1	Manual. Returns the sample menu in 0.4s without touching a model.

We deliberately do *not* fall back on a blown deadline or a malformed request. Neither is fixed by asking someone else, and both would just spend what is left of the budget.

Every failure says something true

SITUATION	STATUS	WHAT THE USER READS
Photo is not a menu	422	“That does not look like a menu. Try another photo.”
All keys rate-limited	429	“We are getting a lot of requests.”
Budget exhausted	504	“That took too long. Try once more.”
Image too large	413	“That photo is too large.”
Model broke the contract twice	200	Sample menu, <i>labelled as sample</i>

The rule: the endpoint never returns a blank 500. And when it degrades to the sample menu, **the screen says so**. Showing someone else's menu as though it were their photo is worse than showing an error — especially to a judge who just photographed their own.

Three layers against a wrong answer

1. A strict response schema, so the model cannot return prose.
2. Runtime validation against the same contract that types the app.
3. A sanity check and one repair pass before anything reaches the screen.

7 · What we measured

All figures from real runs against real photographed menus, not estimates.

Speed

STEP	TIME
Browser downscale, 12.9MB photo → 194KB	~400ms
12-dish Argentine menu, end to end	8–15s
39-dish Spanish menu (capped to 20)	8–16s
<code>?demo=1</code> on production	0.4s
Follow-up question about a dish	0.4s

Choosing the model on evidence

The primary model is `gemini-3.1-flash-lite` because it measured better here, not because it is cheaper. On our hardest menu:

	GEMINI-3.6-FLASH	GEMINI-3.1-FLASH-LITE
Time	14–25s, timing out in production	8–11s
Dishes read	20	20
Language / allergens	correct	correct

Identical output, roughly half the time. The slower model is still the fallback tier, so nothing was lost by demoting it.

Accuracy on real photographs

CHECK	RESULT
39-dish Spanish menu, laminated with glare	All 20 (cap) read, language detected
Gluten-free marked by <i>icon</i> , not text	All 5 correct; pasta and pastry correctly excluded
Pesto → contains nuts	Correct — never stated on the menu
Migas (breadcrumbs) → not gluten-free	Correct
Regional translation, “ <i>Patatas a lo pobre</i> ”	“Poor Man's Potatoes”
Photo of a weather report	Rejected with a plain-English reason

Build discipline

18 MERGED PRS	0 DIRECT PUSHES TO MAIN	100% CI-GATED MERGES	0 CONSOLE ERRORS
-------------------------	-----------------------------------	--------------------------------	----------------------------

`main` is protected. Every merge passes `tsc`, ESLint and a production build — run with `DEMO_MODE=1`, so the build can never depend on a live API key.

8 · Running the demo

Before you walk up

- Phone on venue Wi-Fi *and* mobile data as backup.
- **Load the URL once to warm it.** A cold serverless start adds seconds to a request that already takes ten.
- Use `smartmenu-two-ruddy.vercel.app`. Not any other deployment.
- Printed menu in hand. Screen recording saved locally.

The run — about 90 seconds

1. **Pitch, 15s.** “You are in Rome, the menu is in Italian, and you are keto. Google Translate gives you words. It does not tell you what you can actually eat.”
2. **Scan.** Take the photo live.
3. **Preferences.** Three taps. Say them out loud.
4. **Processing.** Let the stages run. *Do not talk over them* — they are explaining the product for you.
5. **Smart Menu.** “Ranked against my diet. Green I can eat, red I should skip.” Point at the top and the bottom.
6. **Dish details.** Tap the top dish. “Macros, real ingredients, and — this is the part that matters — *why* it scored that way.”
7. **Close, 15s.** “One photo, one model call, no typing. Works on any phone with a browser.”

If it breaks, say nothing about the error. Add `?demo=1` to the URL and keep talking — it answers in under half a second. If that fails, switch to the recording without pausing. **Never debug on stage.**

Known limits — be honest if asked

- Menus longer than 20 dishes are capped at 20, in menu order.
- Nutrition is estimated, not looked up. We sanity-check it and say so.
- Category tiles are gradient-and-icon, not photographs. Generated photos are wired but were blocked by a daily quota.
- State lives in the browser session. Nothing is stored server-side, and there is no account.

9 · Questions we expect

“THE NUTRITION IS JUST GUESSED.”

It is estimated, and we say so on screen. A restaurant does not publish macros; a database lookup on a fuzzy foreign dish name would be an estimate too, just slower. We reject implausible values and re-ask. For *should I order this*, a directionally right estimate beats a precise number for the wrong dish.

“WHAT IF THE MODEL IS WRONG?”

Three layers: a strict schema so it cannot return prose, runtime validation against the same contract that types the app, and a repair pass. And the scoring — the part that gives advice — is not the model at all.

“WHY NOT USE A REAL NUTRITION DATABASE?”

Because the hard part is not the lookup, it is the match. “Trofie al Pesto Genovese” does not appear in any database. You would be fuzzy-matching a foreign dish name to a generic row and calling the result precise.

“IS THIS JUST A WRAPPER AROUND AN API?”

The model call is one file. The rest is the judgement layer: a deterministic scorer that can explain itself, a validation and repair pipeline, a three-tier fallback ladder inside a fixed time budget, and a UI that admits when it is showing sample data. The model is a component, not the product.

“WHAT HAPPENS ON A MENU WITH NOTHING YOU CAN EAT?”

You still get a ranking — ordered by how close each dish is to acceptable, with the reason on every card. That case is specifically handled, because it is the case where a diner most needs help.

“WHY A WEB APP RATHER THAN NATIVE?”

You are holding the demo. No install, no TestFlight, no review. It is React either way; the server side exists only because an API key cannot live in browser JavaScript.

10 · What is next

- **Allergen alerts** rather than score penalties — the flags are already in the schema.
- **Real nutrition data** for the dishes common enough to have it, with the model as fallback for everything else.
- **Restaurant partnerships** — a restaurant uploads its menu once and gets a QR code, so the diner skips the photo entirely. That is also the business model.

One line to leave them with: every other tool translates the menu. SmartMenu reads it, understands what is in the food, and tells you what to order.

Appendix A · The contract

`lib/schema.ts`, in full. Everything downstream is typed from this file — the model request, the API response, the store, the scorer and every component prop. It was frozen before anyone wrote a screen.

```
export const Prefs = z.object({
  diet: z.enum(DIETS).default("none"),
  lowCarb: z.boolean().default(false),
  highProtein: z.boolean().default(false),
  noNuts: z.boolean().default(false),
  targetLang: z.string().default("en"),
});

export const Nutrition = z.object({
  calories: z.number(),
  protein: z.number(),
  carbs: z.number(),
  fat: z.number(),
});

export const Flags = z.object({
  vegan: z.boolean(),
  vegetarian: z.boolean(),
  glutenFree: z.boolean(),
  containsNuts: z.boolean(),
});

export const MenuItem = z.object({
  id: z.string(),
  originalName: z.string(),
  translatedName: z.string(),
  section: z.string(),
  category: z.enum(CATEGORIES),
  ingredients: z.array(z.string()),
  nutrition: Nutrition,
  flags: Flags,
  confidence: z.number().min(0).max(1).default(0.8),
});

export const AnalyzedMenu = z.object({
  isMenu: z.boolean().default(true),
  rejectionReason: z.string().nullable().default(null),
  sourceLang: z.string(),
  items: z.array(MenuItem),
});
```

Why this matters more than it looks. The JSON schema sent to the model is *generated* from these definitions at runtime, not written by hand. Change a field here and the model request follows automatically, so the two can never drift apart.

Appendix B · The prompt

The complete instruction sent with every photograph, from `buildPrompt()` in `lib/llm.ts`. `{targetLang}` is substituted from the user's preferences. This single prompt does OCR, structuring, translation, ingredient inference, macro estimation and allergen flagging in one pass.

You are reading a photograph of a restaurant menu.

Return every dish you can read, in the order it appears on the menu.

For each dish:

- `originalName`: the dish name exactly as printed, including accents. Do not translate this field.
- `translatedName`: a natural, appetising name in en. Translate the meaning, not word for word. "Tagliata di Manzo con Rucola" is "Sliced Beef with Rocket", not "Cut of Beef with Arugula".
- `section`: the menu section this dish sits under, translated into en (for example "Starters", "First courses", "Main courses", "Desserts"). If the menu has no sections, infer the most fitting one.
- `category`: one of pasta, fish, meat, risotto, dessert, salad, other. Pick the closest.
- `ingredients`: the main ingredients, in en. Include what the dish obviously contains even if the menu does not spell it out (carbonara contains egg and pork).
- `nutrition`: your best estimate for one restaurant portion as served. calories in kcal, protein/carbs/fat in grams. A plate of pasta is 600-900 kcal, a green salad is 150-350, a grilled fish main is 350-550. Never return zeros.
- `flags`: vegan, vegetarian, glutenFree, containsNuts. Be careful and be strict:
 - wheat pasta, bread, breaded or battered dishes and most desserts are NOT glutenFree
 - pesto contains pine nuts, so `containsNuts` is true; so are most pastries and anything with almonds or walnuts
 - anything with cheese, butter, cream, egg, honey, fish or meat is NOT vegan
 - anything with fish, shellfish or meat is NOT vegetarian
- `confidence`: 0 to 1, how sure you are about this dish. Lower it when the text is blurry or the dish is unfamiliar.

Also return:

- `sourceLang`: the language the menu is written in, as a short code such as "it", "fr", "es".
- `isMenu`: true if this image really is a food or drink menu.
- `rejectionReason`: null when `isMenu` is true. When the image is not a menu, set `isMenu` to false, return an empty `items` array, and put one short friendly sentence here describing what you actually see, such as "This looks like a photo of a person, not a menu."

Return at most 20 dishes. If the menu lists more than that, return the first 20 in the order they appear and stop.

Read only what is on the menu. Do not invent dishes that are not there.

Appendix B · Notes on the prompt

Four choices in there are worth being able to defend.

Worked examples beat instructions

“Translate naturally” produces literal translations. One concrete example — “*Tagliata di Manzo con Rucola*” is “*Sliced Beef with Rocket*”, not “*Cut of Beef with Arugula*” — changes the register of every dish name in the response. The same trick carries the allergen rules: naming pesto and pine nuts explicitly is what makes the model catch the nut cases the menu never mentions.

Calorie anchors, because the model has no scale

Without “a plate of pasta is 600–900 kcal, a green salad is 150–350”, the estimates drift wildly between dishes on the same menu. The anchors are what make the numbers comparable enough to rank. They are also backed up in code: macros that fall out of range, or that do not add up to the stated calories, are rejected and re-asked once.

The rejection path rides along

`isMenu` and `rejectionReason` are in the same response as the dishes. A photo of a face or a receipt comes back as a friendly sentence in one round trip — no second classifier call, no extra latency on the happy path.

The cap is in the prompt, not just the code

We cap at 20 dishes either way, but asking for 20 rather than trimming 39 afterwards saves the generation time for 19 dishes we would throw away. That single line took a 39-dish Spanish menu from a **504 timeout** to **20 dishes in 11 seconds**.

What is deliberately absent: nothing asks the model to score, rank or advise. It reports what it sees. Every judgement is made afterwards, in code we can read, test and explain.